Sliding Window Counter Algorithm (Rate Limiting)

The **Sliding Window Counter Algorithm** is a **hybrid approach** that combines:

- Fixed Window Counter
- Sliding Window Log

It improves accuracy over fixed windows while using far less memory than storing every request timestamp.

1. Core Idea

Instead of keeping full request logs, the algorithm uses:

```
None
Current window request count

+

Weighted portion of previous window count
```

This gives an **approximate rolling window count**.

More accurate than Fixed Window, cheaper than Sliding Window Log.

2. Why It Is Needed

Fixed Window problem:

```
None
Traffic spikes at window boundaries can bypass limits
```

Sliding Window Log problem:

```
None
Very accurate but stores many timestamps (high memory)
```

Sliding Window Counter balances both.

3. How It Works

Track two counters:

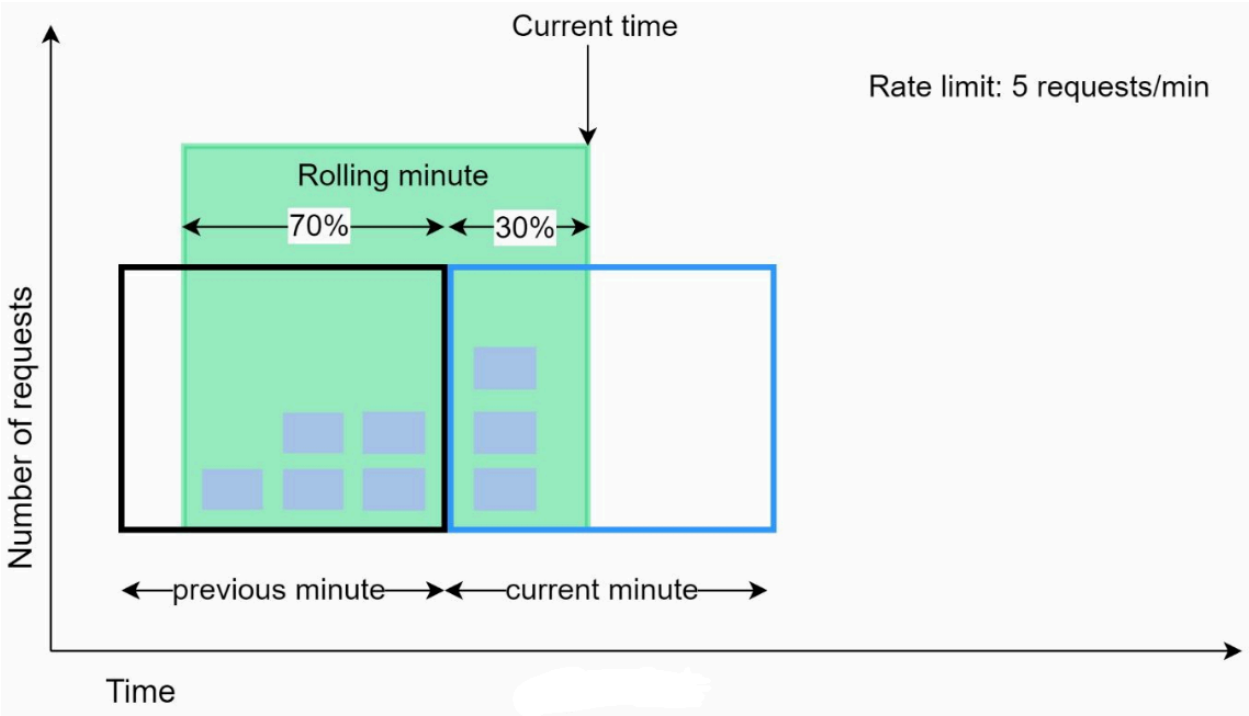
- Previous time window count
- Current time window count

When a request arrives:

```
None
Estimated Count =

Current Window Count

+

(Previous Window Count × Overlap Percentage)
```

If estimated count exceeds limit → reject request.

4. Example



System limit:

```
None
7 requests / minute
```

Traffic:

```
None
Previous minute = 5 requests

Current minute  = 3 requests
```

New request arrives at:

```
None
30% position inside current minute
```

So previous window overlap with rolling minute = 70%

Formula

```
None
Rolling Count =

3 + (5 × 0.7)

= 6.5
```

Rounded down:

```
None
6 requests
```

Since limit is 7:

Current request is allowed

After one more request:

```
None
Limit reached
```

5. Why This Works

Instead of assuming hard reset every minute, it gradually reduces influence of previous window.

```
None
Older requests matter less as time moves forward
```

6. Advantages

✔ Smooths Traffic Spikes

Boundary spikes are reduced.

✔ Memory Efficient

Only need:

```
None
2 counters + timestamp/window metadata
```

No need to store all request timestamps.

✔ Fast Performance

Simple arithmetic + counters.

✔ Practical Accuracy

Used in real production systems.

7. Disadvantages

Approximation Only

Assumes requests in previous window were evenly distributed.

```
None
May slightly overcount or undercount
```

Not Ideal for Ultra-Strict Limits

For security-critical systems, Sliding Window Log may be preferred.

8. Real-World Note

According to Cloudflare experiments:

```
None
Only ~0.003% requests were wrongly allowed or blocked

(out of 400 million requests)
```

Very good trade-off in practice.

9. Comparison of Window Algorithms

Algorithm	Accuracy	Memory	Boundary Spike	Complexity
Fixed Window	Low	Low	High	Very Low
Sliding Log	Very High	High	None	Medium
Sliding Counter	High	Low	Low	Medium

10. Best Use Cases

- Public APIs
- CDN edge throttling
- Large-scale gateways
- SaaS API quotas
- High-throughput systems needing efficiency

11. Implementation Idea

Use Redis or in-memory counters:

```
None
prev_count

curr_count

window_start_time
```

Rotate counters when new window starts.

Mental Model

```
None
Sliding Window Counter = Smart average of current + previous
window traffic
```

One-Line Summary

Sliding Window Counter Algorithm rate-limits traffic using the current request count plus a weighted portion of the previous window count, providing near-accurate throttling with low memory overhead.